UNIVERSIDADE DE MOGI DAS CRUZES

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

JONATHAN AUGUSTO RODRIGUES BARBOSA

LOGIN COMPLETO

Mogi das Cruzes

2026

JONATHAN AUGUSTO RODRIGUES BARBOSA

LOGIN COMPLETO

Documentação técnica do sistema Login Completo, apresentada ao curso de Bacharelado em Sistemas de Informação da Universidade de Mogi das Cruzes, sob orientação do Prof. Alessandro Aparecido da Silva Horas.

Mogi das Cruzes

2026

RESUMO

Este documento apresenta a documentação técnica do sistema Login Completo, uma aplicação web desenvolvida em Java com o framework Spring Boot para o controle de estoque de peças mecânicas, protegida por um mecanismo completo de autenticação e autorização. São descritos a estrutura do sistema, organizada em módulos de negócio com camadas bem definidas e componentes genéricos reutilizáveis; a integração com o banco de dados MongoDB Atlas, incluindo a configuração da conexão por variáveis de ambiente, o modelo de dados, os índices e os mecanismos de auditoria e de controle de concorrência; e as principais decisões de design, como a autenticação baseada em sessão, a hierarquia de perfis de acesso (Estoque, Supervisor e Gerente), o armazenamento de senhas com hash BCrypt e a redefinição de senha por token de uso único enviado por e-mail. Por fim, são apresentadas orientações para futuras adaptações, como a inclusão de novos módulos e perfis e a preparação do sistema para o ambiente de produção.

Palavras-chave: Autenticação; Controle de acesso; Spring Boot; MongoDB Atlas; Thymeleaf.

ABSTRACT

This document presents the technical documentation of the Login Completo system, a web application developed in Java with the Spring Boot framework for managing the inventory of mechanical parts, protected by a complete authentication and authorization mechanism. It describes the system structure, organized into business modules with well-defined layers and reusable generic components; the integration with the MongoDB Atlas database, including the connection setup through environment variables, the data model, the indexes and the auditing and concurrency control mechanisms; and the main design decisions, such as session-based authentication, the hierarchy of access profiles (Stock, Supervisor and Manager), password storage with BCrypt hashing and password reset through a single-use token sent by e-mail. Finally, it provides guidelines for future adaptations, such as adding new modules and profiles and preparing the system for the production environment.

Keywords: Authentication; Access control; Spring Boot; MongoDB Atlas; Thymeleaf.

SUMÁRIO

1. INTRODUÇÃO 6
1.1. OBJETIVOS 6
1.2. VISÃO GERAL DO SISTEMA 6

2. ESTRUTURA DO SISTEMA 8
2.1. CAMADAS DOS MÓDULOS 8
2.2. COMPONENTES GENÉRICOS 9
2.3. PERFIS DE ACESSO 10
2.4. FLUXOS PRINCIPAIS 10
2.5. INTERFACE WEB 11

3. INTEGRAÇÃO COM O MONGODB ATLAS 12
3.1. CONFIGURAÇÃO DA CONEXÃO 12
3.2. MODELO DE DADOS 13
3.3. ÍNDICES 13
3.4. AUDITORIA E CONTROLE DE CONCORRÊNCIA 14
3.5. CONSULTAS 14

4. PRINCIPAIS DECISÕES DE DESIGN 16
4.1. AUTENTICAÇÃO BASEADA EM SESSÃO 16
4.2. ARMAZENAMENTO SEGURO DE SENHAS 16
4.3. HIERARQUIA DE PERFIS DERIVADA DA ENUMERAÇÃO 16
4.4. CADASTRO PÚBLICO COM ESCOLHA DE PERFIL 16
4.5. REDEFINIÇÃO DE SENHA COM TOKEN DE USO ÚNICO 17
4.6. ENCERRAMENTO DE SESSÕES 17
4.7. CONFIGURAÇÃO EXTERNA E SEM SEGREDOS NO CÓDIGO 17
4.8. TRATAMENTO DE ERROS VOLTADO AO USUÁRIO 17

5. ORIENTAÇÕES PARA FUTURAS ADAPTAÇÕES 18
5.1. INCLUSÃO DE UM NOVO MÓDULO 18
5.2. INCLUSÃO OU ALTERAÇÃO DE PERFIS 18
5.3. SUBSTITUIÇÃO DO SERVIÇO DE E-MAIL 18
5.4. IMPLANTAÇÃO EM PRODUÇÃO 19

5.5. VALIDAÇÃO DAS ALTERAÇÕES 19
5.6. MELHORIAS RECOMENDADAS 19

6. CONSIDERAÇÕES FINAIS 20

1. INTRODUÇÃO

O Login Completo é uma aplicação web para o controle de estoque de peças mecânicas cujo foco central é a autenticação e a autorização de usuários. O sistema permite que as pessoas se cadastrem, acessem o sistema com usuário e senha, redefinam a senha por e-mail e operem o estoque de acordo com o perfil de acesso de cada uma.

Este documento descreve a estrutura do sistema, a forma como ele se integra ao banco de dados MongoDB Atlas e as principais decisões de *design* adotadas no desenvolvimento, com o objetivo de servir como referência para a manutenção e para futuras adaptações.

1.1. OBJETIVOS

O objetivo geral deste documento é registrar a arquitetura e o funcionamento do sistema Login Completo, de forma que outros desenvolvedores possam compreendê-lo, mantê-lo e adaptá-lo com segurança. Como objetivos específicos, destacam-se:

- descrever a organização do código-fonte em módulos e camadas;
- explicar a integração com o MongoDB Atlas, da configuração da conexão ao modelo de dados;
- justificar as principais decisões de *design* e de segurança;
- orientar a realização de futuras adaptações.

1.2. VISÃO GERAL DO SISTEMA

O sistema oferece as seguintes funcionalidades:

- cadastro público de usuários, com escolha do perfil de acesso;
- *login* por formulário e *logout*, com proteção contra falsificação de requisições (CSRF);
- armazenamento de senhas com *hash* BCrypt e política de senha forte;
- redefinição de senha por link enviado por e-mail e alteração de senha pelo próprio usuário;
- cadastro, consulta, edição, exclusão e entrada de estoque de peças mecânicas, com busca, filtros e paginação;

- indicadores de itens cadastrados e de itens com estoque abaixo do mínimo.

A aplicação foi construída com Java 17 e Spring Boot 4, utilizando os módulos Spring Web MVC, Spring Security, Spring Data MongoDB, Bean Validation e Spring Mail. A interface é gerada no servidor com Thymeleaf e estilizada com Bootstrap 5, e os dados são persistidos no MongoDB Atlas, serviço de banco de dados em nuvem.

2. ESTRUTURA DO SISTEMA

O código-fonte é organizado por módulo de negócio, e não apenas por camada técnica. Cada módulo reúne tudo o que se refere a um mesmo assunto, o que facilita a localização do código, reduz o acoplamento entre partes independentes e permite incluir novas funcionalidades sem alterar as existentes. O Quadro 1 apresenta a estrutura de pacotes da aplicação.

Quadro 1 – Estrutura de pacotes do sistema

```
com.example.LoginCompleto
├── common/      componentes genéricos reutilizáveis
│   ├── entity/       BaseEntity (identificador, versão e auditoria)
│   ├── service/      ICrudService e AbstractCrudService
│   ├── repository/   MongoQuerySupport (busca e paginação)
│   ├── exception/    BusinessException e ResourceNotFoundException
│   ├── validation/   StrongPassword e IPasswordConfirmation
│   ├── enums/        ILabeledEnum
│   └── web/          erros, mensagens e utilitários de tela
├── infra/       configurações técnicas
│   ├── config/       propriedades, MongoDB, índices e assincronia
│   ├── security/     Spring Security, sessões, senhas e tokens
│   └── mail/         envio de e-mails
├── auth/        login, cadastro e redefinição de senha
├── user/        usuário e perfis de acesso
├── item/        peças mecânicas e controle de estoque
└── home/        página inicial com indicadores
```

Fonte: Elaborado pelo autor (2026).

2.1. CAMADAS DOS MÓDULOS

Dentro de cada módulo, as classes são distribuídas em camadas com responsabilidades bem delimitadas, seguindo o mesmo padrão em todo o projeto:

- `entity`: classes persistidas no banco de dados, anotadas com `@Document` e derivadas de `BaseEntity`;
- `repository`: interfaces que estendem `MongoRepository`, identificadas pelo prefixo I (por exemplo, `IUserRepository`), e extensões customizadas para consultas dinâmicas;

- `service`: regras de negócio e verificação de permissões;
- `controller`: recebem as requisições HTTP, acionam os serviços e definem a página exibida;
- `dto`: objetos que representam os dados dos formulários, com as regras de validação;
- `mapper`: conversão entre DTOs e entidades, incluindo a normalização dos dados.

Os controllers nunca recebem entidades diretamente dos formulários: os dados chegam em DTOs validados e são convertidos pelos mappers. Dessa forma, campos sensíveis, como o identificador, a versão ou a senha, não podem ser alterados por um formulário adulterado, ataque conhecido como *mass assignment*.

2.2. COMPONENTES GENÉRICOS

O módulo `common` reúne componentes que não dependem de nenhuma regra de negócio e podem ser reaproveitados por qualquer módulo novo:

- `BaseEntity`: classe base das entidades, com identificador, número de versão e campos de auditoria (data e autor da criação e da última alteração), preenchidos automaticamente;
- `AbstractCrudService`: implementação única das operações de listagem paginada, busca por identificador, gravação, exclusão e contagem, que os serviços específicos apenas estendem;
- `MongoQuerySupport`: busca textual em vários campos, sem distinção entre maiúsculas e minúsculas, e consultas paginadas;
- `GlobalExceptionHandler` e `FlashMessage`: tratamento padronizado dos erros de negócio e das mensagens de sucesso e de erro exibidas nas telas;
- `GlobalBindingAdvice`: remoção automática dos espaços nas extremidades dos campos de texto dos formulários, exceto senhas;
- `StrongPassword` e `IPasswordConfirmation`: validações reutilizáveis de força e de confirmação de senha;
- `ILabeledEnum`, `PageUrlBuilder` e `DisplayFormatter`: exibição de enumerações, paginação com preservação dos filtros e formatação de datas no fuso horário configurado.

2.3. PERFIS DE ACESSO

O sistema trabalha com três perfis de acesso, definidos na enumeração `UserRole`: Estoque, Supervisor e Gerente. Os perfis são hierárquicos, ou seja, cada perfil herda as permissões dos perfis anteriores. O Quadro 2 resume as permissões de cada perfil sobre os itens do estoque.

Quadro 2 – Permissões por perfil de acesso

Ação	Estoque	Supervisor	Gerente
Consultar itens	Sim	Sim	Sim
Cadastrar item e registrar entrada de estoque	Sim	Sim	Sim
Editar item	Não	Sim	Sim
Excluir item	Não	Não	Sim

Fonte: Elaborado pelo autor (2026).

As permissões são verificadas em três níveis: nos controllers, que protegem o acesso às telas; nos serviços, que protegem as regras de negócio por meio da anotação `@PreAuthorize`; e nas próprias telas, que exibem apenas os botões das ações permitidas ao perfil do usuário. As expressões de permissão ficam centralizadas na classe `Permissions`.

2.4. FLUXOS PRINCIPAIS

No *login*, o usuário informa o nome de usuário e a senha. O Spring Security carrega o usuário por meio da classe `AuthorizationService`, compara a senha informada com o *hash* armazenado e, em caso de sucesso, cria uma sessão no servidor. Usuários desativados não conseguem entrar, e a mensagem de falha é a mesma para usuário inexistente e para senha incorreta, o que impede a descoberta dos usuários cadastrados.

No cadastro, o usuário informa nome, nome de usuário, e-mail, perfil e senha. O nome de usuário e o e-mail são normalizados, sem espaços nas extremidades e em letras minúsculas, e precisam ser únicos; a senha é validada quanto à força e gravada apenas na forma de *hash*.

Na redefinição de senha, o usuário informa o e-mail cadastrado e recebe um link com um *token* aleatório, válido por 30 minutos e de uso único. A resposta exibida é sempre a mesma, exista ou não o e-mail informado, e pedidos repetidos em

menos de um minuto são ignorados. Após a redefinição, todas as sessões abertas do usuário são encerradas.

No controle de estoque, os itens podem ser pesquisados por nome, código, marca ou aplicação, filtrados por categoria ou por estoque baixo e ordenados por diferentes critérios. A entrada de estoque soma uma quantidade ao saldo atual do item, enquanto a edição permite corrigir todos os dados do item, inclusive a quantidade.

2.5. INTERFACE WEB

As páginas são geradas no servidor com Thymeleaf. Os modelos ficam em `src/main/resources/templates`, organizados por módulo, e reaproveitam fragmentos comuns localizados em `templates/fragments`: os *layouts* das páginas internas e públicas, o cabeçalho, o menu, as mensagens, os campos de formulário e a paginação. Assim, uma nova página precisa declarar apenas o seu título e o seu conteúdo.

O Bootstrap é distribuído como *WebJar*, isto é, como uma dependência do Maven servida pela própria aplicação, sem depender de servidores externos. Uma única página de erro (`error.html`) atende a todos os códigos de erro, com mensagens específicas para acesso negado e para página não encontrada.

3. INTEGRAÇÃO COM O MONGODB ATLAS

O MongoDB Atlas é o serviço de banco de dados em nuvem oferecido pela MongoDB. A aplicação se conecta a ele por meio do Spring Data MongoDB, que fornece os repositórios, o mapeamento entre as classes Java e os documentos do banco e o acesso direto pelo `MongoTemplate`.

3.1. CONFIGURAÇÃO DA CONEXÃO

A *string* de conexão do Atlas contém o usuário e a senha do banco de dados e, por isso, nunca é gravada no código-fonte. Ela é lida da variável de ambiente `MONGODB_URI`, referenciada no arquivo `application.properties` pela propriedade `spring.mongodb.uri`. O nome do banco é informado separadamente, pela variável `MONGODB_DATABASE`, o que permite manter bancos distintos para cada ambiente no mesmo *cluster*.

A configuração é dividida em perfis do Spring: `dev`, ativo por padrão, que utiliza o banco `logincompleto_dev` e o servidor de e-mail de testes Mailpit; `prod`, que exige todas as variáveis de ambiente e ativa configurações adicionais de segurança; e `test`, usado pelos testes automatizados. O Quadro 3 apresenta as variáveis de ambiente utilizadas.

Quadro 3 – Variáveis de ambiente

Variável	Obrigatória	Finalidade
MONGODB_URI	Sim	String de conexão do MongoDB Atlas
MONGODB_DATABASE	Não (padrão: logincompleto_dev)	Nome do banco de dados
APP_BASE_URL	Em produção	Endereço público usado nos links enviados por e-mail
MAIL_HOST e MAIL_PORT	Em produção	Servidor de envio de e-mails (SMTP)
MAIL_USERNAME e MAIL_PASSWORD	Em produção	Credenciais do servidor SMTP
MAIL_FROM	Não	Endereço do remetente dos e-mails

Fonte: Elaborado pelo autor (2026).

Para que a conexão funcione, o endereço IP do servidor que executa a aplicação deve estar liberado na seção *Network Access* do Atlas. Recomenda-se,

ainda, que o usuário do banco de dados tenha apenas a permissão `readWrite` sobre os bancos da aplicação, e não permissões administrativas.

3.2. MODELO DE DADOS

Os dados são armazenados em três coleções, descritas no Quadro 4. Como todas as entidades estendem `BaseEntity`, todas também possuem os campos `id`, `version`, `createdAt`, `updatedAt`, `createdBy` e `updatedBy`.

Quadro 4 – Coleções do banco de dados

Coleção	Entidade	Principais campos
users	User	name, username, email, password (hash), role, enabled
items	Item	code, name, description, category, brand, application, quantity, minimumQuantity, price, location
password_reset_tokens	PasswordResetToken	tokenHash, userId, expiresAt

Fonte: Elaborado pelo autor (2026).

O perfil de acesso (`role`) e a categoria dos itens (`category`) são gravados como texto, com o nome da constante da enumeração correspondente. O preço dos itens é gravado no tipo `Decimal128`, o tipo decimal nativo do MongoDB, definido pela propriedade `spring.data.mongodb.representation.big-decimal`. Esse tipo armazena o valor exato, sem os erros de arredondamento dos números de ponto flutuante, e permite ordenar e filtrar por preço.

3.3. ÍNDICES

Os índices são declarados nas próprias entidades, com a anotação `@Indexed`, e criados na inicialização da aplicação pela classe `MongoIndexInitializer`, que percorre todas as entidades mapeadas. Desse modo, os índices de uma nova entidade são criados automaticamente, sem alteração de configuração. O Quadro 5 apresenta os índices existentes.

Quadro 5 – Índices das coleções

Coleção	Campo	Tipo	Finalidade
users	username	Único	Impedir nomes de usuário repetidos
users	email	Único	Impedir e-mails repetidos
items	code	Único	Impedir códigos de item repetidos
items	name e category	Simples	Agilizar ordenação e filtros
password_reset_tokens	tokenHash	Único	Localizar o token do link recebido
password_reset_tokens	userId	Simples	Invalidar os tokens de um usuário
password_reset_tokens	expiresAt	TTL	Excluir automaticamente os tokens vencidos

Fonte: Elaborado pelo autor (2026).

Os índices únicos complementam as verificações feitas nos serviços: mesmo que duas requisições simultâneas tentem gravar o mesmo nome de usuário, o banco de dados recusa a segunda, e o erro é convertido em uma mensagem amigável. O índice TTL (*time to live*) faz com que o próprio MongoDB exclua os *tokens* de redefinição de senha após o vencimento.

3.4. AUDITORIA E CONTROLE DE CONCORRÊNCIA

A auditoria é habilitada pela classe `MongoConfig`. Ao gravar uma entidade, o Spring Data preenche automaticamente as datas de criação e de alteração e, por meio da classe `SecurityAuditorAware`, o nome do usuário autenticado responsável pela operação.

O campo `version`, anotado com `@Version`, implementa o controle otimista de concorrência: se dois usuários editarem o mesmo item ao mesmo tempo, a segunda gravação é recusada e o usuário é avisado de que o item foi alterado por outra pessoa, sem que nenhuma alteração seja perdida silenciosamente. Já a entrada de estoque é feita por uma única operação atômica de incremento (`$inc`) no banco de dados, de modo que entradas simultâneas nunca se sobrescrevem.

3.5. CONSULTAS

As consultas simples são derivadas automaticamente dos nomes dos métodos dos repositórios, como `findByUsername` e `existsByEmail`. As consultas com filtros opcionais, como a busca de itens, são implementadas em extensões

customizadas dos repositórios (`IItemSearchRepository` e `IItemSearchRepositoryImpl`), que montam os critérios com o `MongoTemplate`. O texto digitado na busca é tratado como texto literal, e não como expressão regular, o que evita erros e abusos nas consultas.

4. PRINCIPAIS DECISÕES DE DESIGN

Esta seção apresenta as decisões que mais influenciam o funcionamento e a evolução do sistema, acompanhadas das respectivas justificativas.

4.1. AUTENTICAÇÃO BASEADA EM SESSÃO

Como a interface é gerada no servidor com Thymeleaf, optou-se pela autenticação baseada em sessão do Spring Security, em vez de *tokens* JWT. A sessão é identificada por um *cookie* inacessível ao JavaScript, que em produção trafega apenas por HTTPS, e a proteção contra CSRF é aplicada automaticamente a todos os formulários. O uso de JWT só se justificaria se a aplicação passasse a oferecer uma API para outros clientes, como aplicativos móveis.

4.2. ARMAZENAMENTO SEGURO DE SENHAS

As senhas são armazenadas com o algoritmo BCrypt, por meio do codificador delegante do Spring Security, que grava junto ao *hash* o identificador do algoritmo utilizado (por exemplo, `{bcrypt}`). Essa escolha permite trocar o algoritmo no futuro sem invalidar as senhas existentes. A política de senha exige de 8 a 64 caracteres, com letras e números; o limite superior existe porque o BCrypt considera apenas os primeiros 72 *bytes* da senha.

4.3. HIERARQUIA DE PERFIS DERIVADA DA ENUMERAÇÃO

A hierarquia entre os perfis é gerada a partir da ordem das constantes da enumeração `UserRole`, da menor para a maior permissão. Assim, a inclusão de um novo perfil exige apenas a sua inserção na posição correta da enumeração, e a hierarquia passa a valer automaticamente nos controllers, nos serviços e nas telas.

4.4. CADASTRO PÚBLICO COM ESCOLHA DE PERFIL

Por definição do projeto, não há uma tela de gestão de usuários: qualquer pessoa pode criar uma conta e escolher o próprio perfil de acesso. Essa decisão simplifica a demonstração do controle de acesso, mas permite que qualquer pessoa obtenha o perfil Gerente. Em um ambiente real, recomenda-se que o cadastro atribua sempre o perfil de menor permissão e que a promoção de perfis seja feita por um gerente ou diretamente no banco de dados.

4.5. REDEFINIÇÃO DE SENHA COM TOKEN DE USO ÚNICO

O link de redefinição contém um *token* aleatório de 256 *bits*, e o banco de dados armazena apenas o seu *hash* SHA-256; assim, nem mesmo quem tiver acesso ao banco consegue montar um link válido. O *token* é consumido por uma operação de exclusão atômica, o que garante o uso único mesmo diante de requisições simultâneas, sem a necessidade de transações.

O e-mail é enviado de forma assíncrona, para que o tempo de resposta não revele se o e-mail está cadastrado e para que uma falha no servidor de e-mail não interrompa a navegação. O endereço usado no link vem da configuração `APP_BASE_URL`, e não da requisição, o que impede ataques de manipulação do cabeçalho *Host*.

4.6. ENCERRAMENTO DE SESSÕES

As sessões ativas são registradas em um `SessionRegistry`. Quando a senha de um usuário é redefinida, todas as suas sessões são encerradas; quando o próprio usuário altera a senha, as demais sessões são encerradas e a sessão atual é mantida. Dessa forma, quem eventualmente tenha obtido acesso à conta perde esse acesso assim que a senha é trocada.

4.7. CONFIGURAÇÃO EXTERNA E SEM SEGREDOS NO CÓDIGO

As informações sensíveis ou dependentes do ambiente, como a *string* de conexão do banco, as credenciais de e-mail e o endereço público da aplicação, são lidas de variáveis de ambiente. As configurações próprias da aplicação são agrupadas e validadas na inicialização pela classe `AppProperties`, de modo que uma configuração ausente ou inválida impede a aplicação de iniciar, em vez de causar falhas durante o uso.

4.8. TRATAMENTO DE ERROS VOLTADO AO USUÁRIO

Os erros de validação e de regras de negócio são exibidos no próprio formulário, que mantém os dados já digitados. Os erros inesperados são apresentados em uma página padronizada, sem detalhes técnicos, e as mensagens de conversão de valores, como um texto informado em um campo numérico, foram traduzidas no arquivo `messages.properties`.

5. ORIENTAÇÕES PARA FUTURAS ADAPTAÇÕES

5.1. INCLUSÃO DE UM NOVO MÓDULO

Para incluir um novo cadastro no sistema, recomenda-se seguir os passos abaixo, tomando o módulo `item` como referência:

- criar o pacote do módulo em `com.example.LoginCompleto`, com as camadas `entity`, `repository`, `service`, `controller`, `dto` e `mapper`;
- criar a entidade estendendo `BaseEntity`, anotada com `@Document`, e declarar os índices necessários com `@Indexed`;
- criar o repositório estendendo `MongoRepository` e, se houver busca com filtros, uma extensão customizada que utilize o `MongoQuerySupport`;
- criar o serviço estendendo `AbstractCrudService` e protegê-lo com `@PreAuthorize`, usando as expressões da classe `Permissions`;
- criar o DTO com as validações e o mapper de conversão;
- criar o controller e as páginas, reutilizando o *layout* e os fragmentos de formulário e de paginação;
- adicionar o link do módulo ao menu, no arquivo `fragments/navbar.html`.

5.2. INCLUSÃO OU ALTERAÇÃO DE PERFIS

Um novo perfil deve ser adicionado à enumeração `UserRole` na posição correspondente ao seu nível de permissão, e uma constante equivalente deve ser criada na classe `Permissions`. Nas telas, a exibição dos botões é controlada pelo atributo `sec:authorize`, que também respeita a hierarquia automaticamente.

5.3. SUBSTITUIÇÃO DO SERVIÇO DE E-MAIL

O envio de e-mails depende apenas da interface `IMailService`. Para utilizar outro provedor, basta configurar as variáveis `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME` e `MAIL_PASSWORD` com os dados de um servidor SMTP, ou criar uma nova implementação da interface, por exemplo para a API de um serviço de e-mail transacional. Os modelos das mensagens ficam em `templates/mail`.

5.4. IMPLANTAÇÃO EM PRODUÇÃO

Para a implantação, deve-se ativar o perfil `prod` (variável `SPRING_PROFILES_ACTIVE=prod`), definir todas as variáveis de ambiente obrigatórias e publicar a aplicação com HTTPS, pois nesse perfil o *cookie* de sessão é enviado apenas em conexões seguras. Recomenda-se também utilizar um banco de dados exclusivo de produção no Atlas, acessado por um usuário com permissões restritas, e liberar no *Network Access* apenas o endereço do servidor.

5.5. VALIDAÇÃO DAS ALTERAÇÕES

O projeto possui uma suíte de testes automatizados, executada com o comando `./mvnw test`, que combina testes unitários e testes de integração. Os testes de integração utilizam o Testcontainers para criar um MongoDB temporário no Docker, sem depender do Atlas, e verificam o *login*, o cadastro, a redefinição de senha, as permissões de cada perfil e as regras de estoque. Recomenda-se executar a suíte após cada adaptação e incluir novos testes para as funcionalidades adicionadas.

5.6. MELHORIAS RECOMENDADAS

- restringir o cadastro público ao perfil Estoque e criar uma funcionalidade de promoção de perfis;
- limitar o número de tentativas de *login*, para dificultar ataques de força bruta;
- armazenar as sessões em um repositório compartilhado, caso a aplicação passe a ser executada em mais de uma instância;
- registrar o histórico das movimentações de estoque, além do saldo atual.

6. CONSIDERAÇÕES FINAIS

O sistema Login Completo foi estruturado de forma modular, com componentes genéricos reutilizáveis e regras de segurança aplicadas em várias camadas. A integração com o MongoDB Atlas é configurada inteiramente por variáveis de ambiente e apoiada por índices, auditoria e controle de concorrência, o que contribui para a consistência dos dados.

As decisões de *design* e as orientações descritas neste documento buscam facilitar a manutenção e a evolução do sistema, permitindo que novos módulos, perfis e integrações sejam incluídos seguindo os mesmos padrões já adotados.